

dtd详解

1.DTD 简介

文档类型定义（DTD）可定义合法的XML文档构建模块。它使用一系列合法的元素来定义文档的结构。

DTD 可被成行地声明于 XML 文档中，也可作为一个外部引用。

1). 内部的 DOCTYPE 声明

假如 DTD 被包含在您的 XML 源文件中，它应当通过下面的语法包装在一个 DOCTYPE 声明中：

```
<!DOCTYPE root-element [element-declarations]>
```

带有 DTD 的 XML 文档实例（请在 IE5 以及更高的版本打开，并选择查看源代码）：

```
<?xml version="1.0"?>
<!DOCTYPE note [
<!ELEMENT note (to,from,heading,body)>
<!ELEMENT to (#PCDATA)>
<!ELEMENT from (#PCDATA)>
<!ELEMENT heading (#PCDATA)>
<!ELEMENT body (#PCDATA)>
]>
<note>
<to>Tove</to>
<from>Jani</from>
<heading>Reminder</heading>
<body>Don't forget me this weekend</body>
</note>
```

2). 外部文档声明

假如 DTD 位于 XML 源文件的外部，那么它应通过下面的语法被封装在一个 DOCTYPE 定义中：

```
<!DOCTYPE root-element SYSTEM "filename">    // dtd文件在本地服务器
<!DOCTYPE root-element PUBLIC "filename">    // dtd文件在远程服务器
```

这个 XML 文档和上面的 XML 文档相同，但是拥有一个外部的 DTD：

```
<?xml version="1.0"?>
<!DOCTYPE note SYSTEM "note.dtd">
<note>
  <to>Tove</to>
  <from>Jani</from>
  <heading>Reminder</heading>
  <body>Don't forget me this weekend!</body>
</note>
```

这是包含 DTD 的 "note.dtd" 文件：

```
<!ELEMENT note (to,from,heading,body)>
<!ELEMENT to (#PCDATA)>
<!ELEMENT from (#PCDATA)>
<!ELEMENT heading (#PCDATA)>
<!ELEMENT body (#PCDATA)>
```

3).为什么使用 DTD?

通过 DTD，您的每一个 XML 文件均可携带一个有关其自身格式的描述。

通过 DTD，独立的团体可一致地使用某个标准的 DTD 来交换数据。

而您的应用程序也可使用某个标准的 DTD 来验证从外部接收到的数据。

您还可以使用 DTD 来验证您自身的数据。

2. XML 文档构建模块

所有的 XML 文档（以及 HTML 文档）均由以下简单的构建模块构成：

- 元素
- 属性
- 实体
- PCDATA
- CDATA

1).元素

元素是 XML 以及 HTML 文档的**主要构建模块**。

HTML 元素的例子是 "body" 和 "table"。XML 元素的例子是 "note" 和 "message"。元素可包含文本、其他元素或者是空的。空的 HTML 元素的例子是 "hr"、"br" 以及 "img"。

在一个 DTD 中，元素通过元素声明来进行声明。

在 DTD 中，XML 元素通过元素声明来进行声明。元素声明使用下面的语法：

```
<!ELEMENT element-name category>
或
<!ELEMENT element-name (element-content)>
空元素通过类别关键词EMPTY进行声明
只有 PCDATA 的元素通过圆括号中的 #PCDATA 进行声明
通过类别关键词 ANY 声明的元素，可包含任何可解析数据的组合
... //原宿申明类型 https://www.runoob.com/dtd/dtd-elements.html
```

2).属性可提供有关元素的额外信息。

属性总是被置于某元素的开始标签中。属性总是以**名称/值**的形式成对出现的。

在 DTD 中，属性通过 ATTLIST 声明来进行声明。

声明属性

属性声明使用下列语法：

```
<!ATTLIST element-name attribute-name attribute-type attribute-value>
DTD 实例：
<!ATTLIST payment type CDATA "check">
XML 实例：
<payment type="check" />
```

以下是 **属性类型**的选项：

类型	描述
CDATA	值为字符数据 (character data)
(<i>en1</i> <i>en2</i> ..)	此值是枚举列表中的一个值
ID	值为唯一的 id
IDREF	值为另外一个元素的 id
IDREFS	值为其他 id 的列表
NMTOKEN	值为合法的 XML 名称
NMTOKENS	值为合法的 XML 名称的列表
ENTITY	值是一个实体
ENTITIES	值是一个实体列表
NOTATION	此值是符号的名称
xml:	值是一个预定义的 XML 值

默认**属性值**可使用下列值：

值	解释
值	属性的默认值
#REQUIRED	属性值是必需的
#IMPLIED	属性不是必需的
#FIXED value	属性值是固定的

//在 ie浏览器中 #REQUIRED 和#FIXED value 的效果相反

3). 实体

实体是用来定义普通文本的变量。实体引用是对实体的引用。

大多数同学都了解这个 HTML 实体引用：" "。这个"无折行空格"实体在 HTML 中被用于在某个文档中插入一个额外的空格。

当文档被 XML 解析器解析时，实体就会被展开。

实体是用于定义引用普通文本或特殊字符的快捷方式的变量。

- 实体引用是对实体的引用。
- 实体可在内部或外部进行声明。

内部实体声明

语法

```
<!ENTITY entity-name "entity-value">
```

实例

```
DTD 实例：
<!ENTITY writer "Donald Duck.">
<!ENTITY copyright "Copyright runoob.com">
XML 实例：
<author>&writer;&copyright;</author>
```

注意：一个实体由三部分构成：一个和号 (&)，一个实体名称，以及一个分号 (;)。调用该实体时，不再使用&，改为使用%，其他不变。

外部实体声明

语法

```
<!ENTITY entity-name SYSTEM "URI/URL">
```

实例

DTD 实例：

```
<!ENTITY writer SYSTEM "http://www.runoob.com/entities.dtd">
<!ENTITY copyright SYSTEM "http://www.runoob.com/entities.dtd">
```

XML example:

```
<author>&writer;&copyright;</author>
```

4). PCDATA

PCDATA 的意思是被解析的字符数据（parsed character data）。

可把字符数据想象为 XML 元素的开始标签与结束标签之间的文本。

PCDATA 是会被解析器解析的文本。这些文本将被解析器检查实体以及标记。

文本中的标签会被当作标记来处理，而实体会被展开。

不过，被解析的字符数据不应当包含任何 &、< 或者 > 字符；需要使用 &、< 以及 > 实体来分别替换它们。

5). CDATA

CDATA 的意思是字符数据（character data）。

CDATA 是不会被解析器解析的文本。在这些文本中的标签不会被当作标记来对待，其中的实体也不会被展开。